

## Task 8: LangChain and RAG

Rami Al Habash 3/12/2026

CMPM 118 Dr. Leilani Gilpin

### Goal

1. Reimplement LINC using LangChain
2. Use RAG to query the knowledge base to give models additional context
3. Use LangChain with logical inference

### Approach

This is LINC's original pipeline: ProofWriter  $\rightarrow$  Semantic Parser (LLM)  $\rightarrow$  FOL  $\rightarrow$  Prover9  $\rightarrow$  results (label vs. prediction).

This is my pipeline (LINC+): Prolog  $\rightarrow$  LLM-generated ProofWriter  $\rightarrow$  RAG  $\rightarrow$  Semantic Parser (LLM)  $\rightarrow$  Prover9  $\rightarrow$  Prover9 trace and results.

$\rightarrow$  To view the queries/conclusions and their trace, check out "query\_results.txt": 

### LangChain and LINC

I took my time learning LangChain and its libraries. I primarily used LangChain to chain the prompt, LLM, and output. This chain really simplified a lot of code. I am a big fan of LangChain and will be using it for a while. I enjoyed reimplementing LINC using it.

### Using RAG and Querying

For this part of the task, I decided to use FAISS. I spent some time understanding how FAISS works. It is somewhat similar to k-nearest neighbors. I loaded the generated ProofWriter knowledge base into the vector store and then used a nearest-neighbor search to find similar ProofWriter examples. Afterwards, I gave the semantic parser LLM similar ProofWriter examples to show it the general structure of ProofWriter examples and to expand its domain.

### Using Langchain With Logical Inference

I decided to use LangChain to help explain logical inference. Prover9 uses neither backwards nor forward chaining. Prover9 uses resolution-based inference. Prover9 does a proof by contradiction. To get an inference trace, I used nltk's wrapper `_call_prover9()`. The wrapper captures an stdout trace of Prover9's inference. Afterwards, for readability, I used LangChain to invoke an LLM chain that translates the trace into natural language.

### Further Improvements

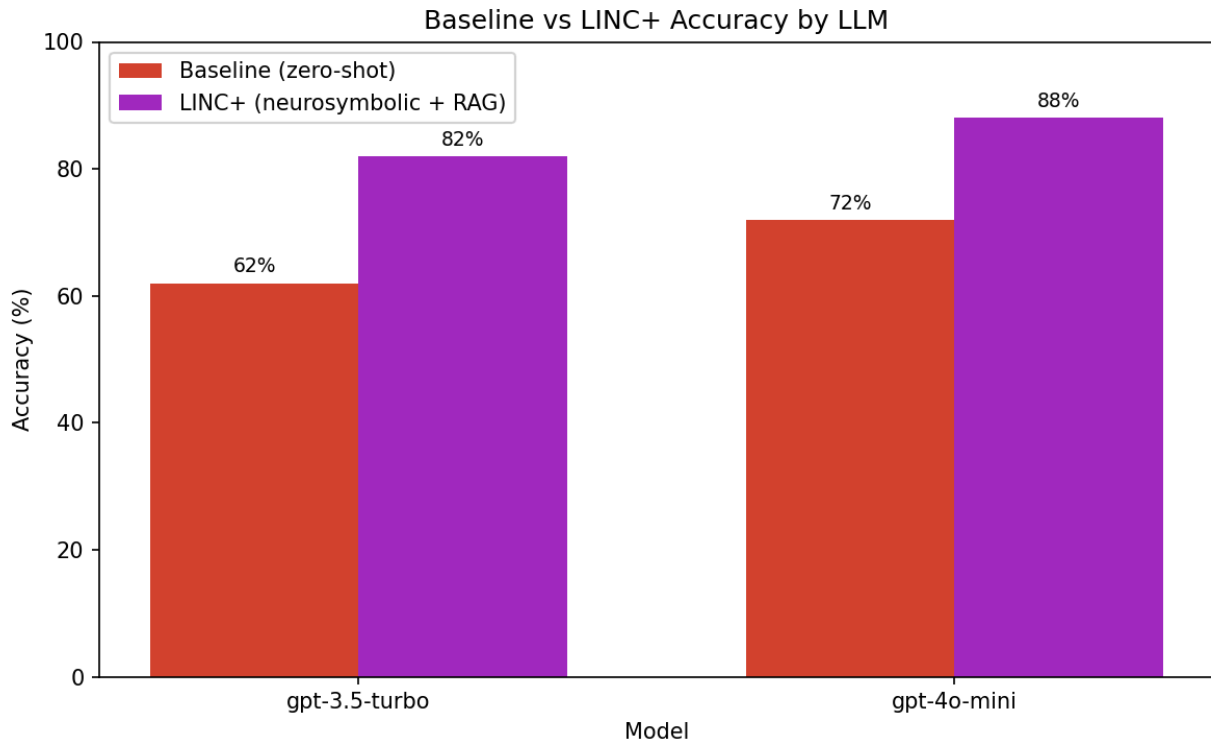
I have an idea for a better use of RAG that might deliver better results than the LINC paper. I will test it out in task 9. LINC shows the semantic parser LLM a random set of FOLIO (an NL and FOL dataset) examples to help it translate NL to FOL. I would like to use RAG to show the semantic parser LLM similar FOLIO examples to the example it's currently facing. I will translate my KB into ProofWriter and FOLIO examples to start with.

## Results

Querying/testing conclusions: please find “results.txt.”

Accuracy (results.png) (not really discussed too much above):

I will use RAG on my KB’s FOLIO samples in task 9, then compare it with this graph that uses ProofWriter examples. We should see much better results.



## References

### Papers

LINC's study: <https://arxiv.org/abs/2310.15164>

ProofWriter: <https://arxiv.org/abs/2012.13048>

### AI-Assisted Coding

Used Claude to:

- Better understand LINC's preexisting code in more detail
- Searching through library functions (e.g., from NLTK, Prover9, and LangChain)
- Assist with debugging where appropriate

I critically evaluated all outputs. I wrote all the code or got it from LINC and made changes. I also attributed LINC where appropriate.

→ To understand the code better, refer to the [README.md](#) file.